

Sprint 1 Structured Notes: Web Language Roles and First JavaScript Tools

0. What Sprint 1 Is About

Sprint 1 is the first step in learning JavaScript for the web.

The main goal is to understand:

1. What JavaScript does on a web page.
2. How JavaScript works with HTML and CSS.
3. How to use your first JavaScript tools:
4. `console.log()`
5. comments
6. semicolons
7. button clicks
8. `getElementById()`
9. `addEventListener()`
10. `alert()`
11. `textContent`
12. How to build a tiny interactive web page.
13. How to spot a very common beginner error when JavaScript cannot find an HTML element.

By the end of Sprint 1, you should be able to create a simple HTML page with a button and make JavaScript respond when the button is clicked.

1. The Three Main Web Languages

A basic web page usually uses three languages:

1. HTML
2. CSS
3. JavaScript

They work together, but they do different jobs.

1.1 HTML: Structure

HTML gives the page its structure.

It creates things like:

- headings
- paragraphs
- buttons
- images
- links
- forms
- lists

Example:

```
<h1>Hello</h1>
<p>This is a paragraph.</p>
<button>Click me</button>
```

This code creates content on the page.

HTML answers the question:

What is on the page?

So, HTML is like the skeleton of a website.

1.2 CSS: Style

CSS controls how the page looks.

It can change:

- colors
- font sizes
- spacing
- borders
- layout
- background colors

Example:

```
h1 {  
  color: green;  
}
```

This makes the heading green.

CSS answers the question:

How should the page look?

So, CSS is like the clothing or design of a website.

1.3 JavaScript: Behavior

JavaScript makes the page do things.

It can control:

- button clicks
- alert messages
- menu opening and closing
- form behavior
- text changes
- user interactions
- dynamic page updates

Example idea:

```
When the user clicks the button, show a message.
```

JavaScript answers the question:

What should happen on the page?

So, JavaScript is like the action or behavior of a website.

1.4 Simple Comparison

Language	Job	Simple Meaning
HTML	Structure	What exists on the page

Language	Job	Simple Meaning
CSS	Style	How it looks
JavaScript	Behavior	What happens

Example:

```
<h1>Hello, World!</h1>  
<button onclick="alert('Button clicked!')">Click me</button>
```

In this example:

- `<h1>` creates a heading.
- `<button>` creates a button.
- `onclick="alert('Button clicked!')"` makes something happen when the button is clicked.

2. What JavaScript Adds to a Web Page

HTML and CSS can create a page, but without JavaScript the page is mostly static.

Static means it does not react much.

JavaScript makes a page interactive.

For example:

- A user clicks a button.
- JavaScript notices the click.
- JavaScript runs some code.
- The page responds.

Possible responses:

```
Show an alert.  
Print a message in the console.  
Change text on the page.  
Open a menu.  
Hide something.  
Show something.
```

In Sprint 1, the most important idea is:

JavaScript lets the page respond to the user.

3. Your First HTML Page Structure

A simple HTML page can look like this:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>JavaScript Practice</h1>
    <p>Click the button to run JavaScript.</p>
    <button>Run</button>
  </body>
</html>
```

Let's understand each part.

3.1 <!DOCTYPE html>

```
<!DOCTYPE html>
```

This tells the browser:

This is an HTML document.

You usually put it at the top of an HTML file.

3.2 <html>

```
<html>
  ...
</html>
```

This is the main container for the whole HTML page.

Everything on the page goes inside it.

3.3 <body>

```
<body>
...
</body>
```

The `<body>` contains the visible content of the page.

For example:

- headings
 - paragraphs
 - buttons
 - images
-

3.4 <h1>

```
<h1>JavaScript Practice</h1>
```

This creates a main heading.

`h1` usually means the most important heading on the page.

3.5 <p>

```
<p>Click the button to run JavaScript.</p>
```

This creates a paragraph.

Paragraphs are used for normal text.

3.6 <button>

```
<button>Run</button>
```

This creates a button.

A button is useful because users can click it.

But by itself, a button does not do much. JavaScript gives it behavior.

4. The `<script>` Tag

JavaScript can be written inside a `<script>` tag.

Example:

```
<script>
  console.log("Hello from JavaScript");
</script>
```

The browser reads and runs the JavaScript inside the `<script>` tag.

A full example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Hello</h1>

    <script>
      console.log("Page loaded");
    </script>
  </body>
</html>
```

When the page loads, JavaScript runs this line:

```
console.log("Page loaded");
```

5. Reading Code From Top to Bottom

In Sprint 1, you should practice reading code from top to bottom.

Example:

```
console.log("Start");  
console.log("Middle");  
console.log("End");
```

JavaScript usually runs these lines in order:

```
Start  
Middle  
End
```

This is important because the order of your code can affect what happens.

Before running code, you should try to predict the output.

Good beginner workflow:

1. Read the code.
2. Predict what will happen.
3. Run the code.
4. Compare your prediction with the result.
5. Fix any mistakes.

6. `console.log()`

`console.log()` is one of the first JavaScript tools you should learn.

It prints a message or value in the browser console.

Example:

```
console.log("Page loaded");
```

This prints:

```
Page loaded
```

6.1 What Is the Console?

The console is a developer tool inside the browser.

It is not normally visible to regular users.

Developers use it to:

- test code
 - check values
 - see messages
 - find errors
 - debug problems
-

6.2 `console.log()` Does Not Print on the Web Page

This is important:

```
console.log("Hello");
```

This does not show `Hello` directly on the webpage.

It shows `Hello` in the browser console.

To open the console, you can usually use:

```
Right click page → Inspect → Console
```

or:

```
Ctrl + Shift + J
```

The exact shortcut may depend on your browser or computer.

6.3 Why `console.log()` Is Useful

`console.log()` helps you check if your code is running.

Example:

```
console.log("Page loaded");  
console.log("Button ready");  
console.log("Button clicked");
```

These messages help you understand what is happening step by step.

For example:

- If you see `Page loaded`, your script is connected.
- If you see `Button ready`, JavaScript found the button.
- If you see `Button clicked`, your click event worked.

7. Comments

Comments are notes inside your code.

JavaScript ignores comments when the program runs.

Comments are for humans, not for the browser.

7.1 Single-Line Comments

A single-line comment starts with `//`.

Example:

```
// This is a comment  
console.log("Hello");
```

JavaScript ignores this part:

```
// This is a comment
```

But JavaScript runs this part:

```
console.log("Hello");
```

7.2 Why Comments Are Useful

Comments help you explain your code.

They are useful when:

- you want to remember why you wrote something
- another developer reads your code
- you are learning and want to label sections
- the code is not obvious

Example:

```
// Check that the script is connected correctly  
console.log("Page loaded");
```

This comment explains why the log exists.

7.3 Good Comments vs Weak Comments

Weak comment:

```
// This logs hello  
console.log("Hello");
```

This is not very useful because the code already shows that it logs hello.

Better comment:

```
// Test that JavaScript is running after the page loads  
console.log("Hello");
```

This is better because it explains the purpose.

Good comments usually explain:

Why is this code here?

Not only:

What does this line do?

7.4 Comments in HTML

HTML comments look different from JavaScript comments.

Example:

```
<!-- HTML goal: show a heading, paragraph, and button -->
```

HTML comments are also ignored by the browser.

They are notes for people reading the HTML.

8. Semicolons

A semicolon is written like this:

```
;
```

In JavaScript, semicolons often mark the end of a statement.

Example:

```
console.log("Hello");
```

The semicolon tells JavaScript:

This instruction ends here.

8.1 Statement

A statement is one instruction in JavaScript.

Example:

```
console.log("Start");  
console.log("Stop");
```

There are two statements here:

1. Print `Start` .
2. Print `Stop` .

Each statement ends with a semicolon.

8.2 Do You Always Need Semicolons?

JavaScript can sometimes add semicolons automatically.

This is called automatic semicolon insertion.

But as a beginner, it is safer and clearer to write semicolons yourself.

Recommended beginner habit:

```
console.log("Hello");  
console.log("Learning JavaScript");
```

Semicolons make your code easier to read and can help avoid confusing situations.

9. The `id` Attribute

An `id` gives an HTML element a unique name.

Example:

```
<button id="runBtn">Run</button>
```

Here, the button's ID is:

```
runBtn
```

JavaScript can use this ID to find the button.

Important rule:

IDs should be unique on the page.

This means you should not use the same ID for many elements.

10. Finding an HTML Element with `getElementById()`

JavaScript can find an HTML element by its ID using:

```
document.getElementById("idName");
```

Example HTML:

```
<button id="runBtn">Run</button>
```

Example JavaScript:

```
const button = document.getElementById("runBtn");
```

This means:

Find the HTML element with the ID `runBtn` and store it in a variable called `button`.

So JavaScript finds this element:

```
<button id="runBtn">Run</button>
```

10.1 What Is `document`?

`document` represents the web page.

When you write:

```
document.getElementById("runBtn");
```

You are telling JavaScript:

Search the current HTML document for an element with this ID.

You do not need to fully understand the DOM yet. For Sprint 1, remember:

`document` lets JavaScript access the web page.

10.2 Why the ID Must Match Exactly

The ID in JavaScript must match the ID in HTML.

This works:

```
<button id="runBtn">Run</button>
```

```
const button = document.getElementById("runBtn");
```

This does not work:

```
<button id="runBtn">Run</button>
```

```
const button = document.getElementById("wrongId");
```

Why?

Because there is no element with this ID:

```
wrongId
```

So JavaScript cannot find anything.

11. `const` in Sprint 1 Code

You may see this line:

```
const button = document.getElementById("runBtn");
```

For Sprint 1, you do not need to deeply study variables yet. That comes later.

For now, understand this simple meaning:

Store the found button inside a name called `button`.

So later we can write:

```
button.addEventListener(...);
```

Instead of searching for the button again.

12. Events

An event is something that happens on the page.

Examples of events:

- the user clicks a button
- the user types in an input
- the page loads
- the user moves the mouse
- the user submits a form

In Sprint 1, the main event is:

```
click
```

A click event happens when the user clicks something.

13. `addEventListener()`

`addEventListener()` lets JavaScript listen for an event.

Example:

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

This means:

When the button is clicked, run the code inside the curly braces.

13.1 Parts of `addEventListener()`

Look at this code:

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

Part 1:

```
button
```

This is the element we are listening to.

Part 2:

```
"click"
```

This is the event we are waiting for.

Part 3:

```
() => {  
  console.log("Button clicked");  
}
```

This is the code that runs after the click happens.

Simple sentence:

Listen for a click on the button, then run this code.

14. Arrow Function Used in Click Events

You will see this pattern:

```
() => {  
  console.log("Button clicked");  
}
```

This is an arrow function.

For Sprint 1, think of it as:

A small block of code that runs later.

It runs when the button is clicked.

You do not need to master functions yet. Just recognize the pattern.

15. alert()

`alert()` shows a popup message in the browser.

Example:

```
alert("Button clicked!");
```

When this runs, the browser shows a popup with the message:

Button clicked!

15.1 Difference Between `console.log()` and `alert()`

Tool	Where the message appears	Main use
<code>console.log()</code>	Browser console	Developer testing/debugging
<code>alert()</code>	Popup on the page	Showing a message to the user

Example:

```
console.log("Button clicked");  
alert("You clicked the button!");
```

This does two things:

1. Prints a message for the developer.
2. Shows a popup to the user.

16. Complete Sprint 1 Starter Example

Create a file called:

```
index.html
```

Then write this code:

```
<!DOCTYPE html>
<html>
  <body>
    <!-- HTML goal: show a heading, paragraph, and button -->
    <h1>JavaScript Practice</h1>
    <p id="message">Click the button to run JavaScript.</p>
    <button id="runBtn">Run</button>

    <script>
      // JavaScript goal: react when the user clicks the button
      console.log("Page loaded");

      const button = document.getElementById("runBtn");

      console.log("Button ready");

      button.addEventListener("click", () => {
        console.log("Button clicked");
        alert("You clicked the button!");
      });
    </script>
  </body>
</html>
```

17. Understanding the Complete Example

17.1 HTML Heading

```
<h1>JavaScript Practice</h1>
```

This shows the main heading on the page.

17.2 Paragraph

```
<p id="message">Click the button to run JavaScript.</p>
```

This shows paragraph text.

It also has an ID:

```
id="message"
```

This means JavaScript can find this paragraph later if needed.

17.3 Button

```
<button id="runBtn">Run</button>
```

This creates a button with the text `Run`.

Its ID is:

```
runBtn
```

JavaScript uses that ID to find it.

17.4 Page Loaded Log

```
console.log("Page loaded");
```

This prints a message when the script runs.

It helps you check that JavaScript is connected.

17.5 Finding the Button

```
const button = document.getElementById("runBtn");
```

This finds the button and stores it as `button`.

17.6 Button Ready Log

```
console.log("Button ready");
```

This tells you that JavaScript reached this point.

It also suggests that the button was found successfully.

17.7 Listening for the Click

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
  alert("You clicked the button!");  
});
```

This waits for the user to click the button.

When the click happens:

1. The console shows `Button clicked`.
 2. The browser shows an alert popup.
-

18. Expected Output of the Starter Example

When the page first loads, the console should show:

```
Page loaded  
Button ready
```

When you click the button, the console should show:

```
Button clicked
```

The browser should also show an alert popup:

```
You clicked the button!
```

19. Common Beginner Error: Wrong ID

This is a very important Sprint 1 debugging idea.

Suppose your HTML has this button:

```
<button id="runBtn">Run</button>
```

But your JavaScript says:

```
const button = document.getElementById("wrongId");
```

This is a mistake because the ID does not match.

JavaScript searches for:

```
wrongId
```

But the actual ID is:

```
runBtn
```

So JavaScript finds nothing.

19.1 What Happens When JavaScript Finds Nothing?

When `getElementById()` cannot find an element, it returns:

```
null
```

`null` means:

Nothing was found.

So this line:

```
const button = document.getElementById("wrongId");
```

basically becomes:

```
const button = null;
```

Then this line fails:

```
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

Why?

Because you are trying to do this:

```
null.addEventListener(...)
```

But `null` is not a button.

`null` cannot listen for clicks.

19.2 How to Fix the Wrong ID Error

Make sure the ID in JavaScript matches the ID in HTML exactly.

Correct HTML:

```
<button id="runBtn">Run</button>
```

Correct JavaScript:

```
const button = document.getElementById("runBtn");
```

Also check:

- spelling
- capital letters
- extra spaces
- missing quotes

These are different:

```
runBtn  
RunBtn  
runbtn  
runBtn
```

JavaScript is strict about exact matches.

20. Mini-Build: Start and Stop Buttons

This mini-build teaches you how to connect two buttons to two different actions.

Code:

```
<!DOCTYPE html>  
<html>  
  <body>  
    <h1>Start / Stop Practice</h1>  
  
    <button id="startBtn">Start</button>
```

```
<button id="stopBtn">Stop</button>

<script>
  console.log("Page loaded");

  const startButton = document.getElementById("startBtn");
  const stopButton = document.getElementById("stopBtn");

  startButton.addEventListener("click", () => {
    console.log("Start");
  });

  stopButton.addEventListener("click", () => {
    console.log("Stop");
  });
</script>
</body>
</html>
```

20.1 What This Mini-Build Teaches

This teaches that each button can have its own event listener.

The Start button uses this:

```
startButton.addEventListener("click", () => {
  console.log("Start");
});
```

The Stop button uses this:

```
stopButton.addEventListener("click", () => {
  console.log("Stop");
});
```

Expected behavior:

- Click Start → console shows `Start`
- Click Stop → console shows `Stop`

21. Changing Text on the Page with `textContent`

JavaScript can also change what the user sees on the page.

One way to change text is with:

```
textContent
```

Example:

```
paragraph.textContent = "The button was clicked!";
```

This means:

Change the paragraph's text to `The button was clicked!`.

21.1 Text Change Example

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Text Change Practice</h1>

    <p id="statusText">Waiting...</p>
    <button id="changeBtn">Change Text</button>

    <script>
      const paragraph = document.getElementById("statusText");
      const button = document.getElementById("changeBtn");

      button.addEventListener("click", () => {
        paragraph.textContent = "The button was clicked!";
        console.log("Paragraph text changed");
      });
    </script>
  </body>
</html>
```

21.2 How the Text Change Example Works

This line finds the paragraph:

```
const paragraph = document.getElementById("statusText");
```

This line finds the button:

```
const button = document.getElementById("changeBtn");
```

This listens for a click:

```
button.addEventListener("click", () => {  
  ...  
});
```

This changes the visible paragraph text:

```
paragraph.textContent = "The button was clicked!";
```

This prints a developer message:

```
console.log("Paragraph text changed");
```

Expected behavior:

Before clicking:

```
Waiting...
```

After clicking:

```
The button was clicked!
```

22. Inline Click Handler vs `addEventListener()`

You may see button click code written directly in HTML:

```
<button onclick="alert('Button clicked!')">Click me</button>
```

This works.

It means:

When this button is clicked, show an alert.

But in practice, you will often use `addEventListener()` instead:

```
<button id="runBtn">Click me</button>

<script>
  const button = document.getElementById("runBtn");

  button.addEventListener("click", () => {
    alert("Button clicked!");
  });
</script>
```

Why is `addEventListener()` useful?

Because it keeps JavaScript behavior in JavaScript instead of mixing too much code into HTML.

For Sprint 1, understand both patterns:

1. Inline handler:

```
<button onclick="alert('Button clicked!')">Click me</button>
```

1. JavaScript event listener:

```
button.addEventListener("click", () => {
  alert("Button clicked!");
});
```

23. Practice Goal for Sprint 1

The practice goal is:

Turn Sprint 1 into working code, not only notes.

This means you should not only read the examples.

You should:

1. Type the code yourself.
 2. Predict the result.
 3. Run the page in the browser.
 4. Open the console.
 5. Click the buttons.
 6. Fix errors.
 7. Explain what happened in simple words.
-

24. Warm-Up Drills

Drill 1: Create Basic HTML

Create an HTML file with:

- one heading
- one paragraph
- one button

Example:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>My First JavaScript Page</h1>
    <p>This page will use JavaScript.</p>
    <button id="runBtn">Run</button>
  </body>
</html>
```

Drill 2: Print Three Console Messages

Use `console.log()` to print:

```
Page loaded
Button ready
Button clicked
```

Example:

```
console.log("Page loaded");
console.log("Button ready");
console.log("Button clicked");
```

Drill 3: Add Comments

Add one HTML comment:

```
<!-- HTML goal: show a heading, paragraph, and button -->
```

Add one JavaScript comment:

```
// JavaScript goal: react when the user clicks the button
```

25. Debug Challenge

The debug challenge is:

Change `getElementById("runBtn")` to `getElementById("wrongId")` and explain why `button.addEventListener` fails.

Correct code:

```
const button = document.getElementById("runBtn");
```

Broken code:

```
const button = document.getElementById("wrongId");
```

Explanation:

- HTML has `id="runBtn"`.
- JavaScript searches for `wrongId`.
- No element has that ID.
- JavaScript returns `null`.
- `null` does not have `addEventListener()`.
- So the code fails.

Simple answer:

The ID is wrong, so JavaScript cannot find the button.

26. Mini-Build Task

Task:

Make two buttons. One logs `Start` and the other logs `Stop`.

Solution:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Start and Stop</h1>

    <button id="startBtn">Start</button>
    <button id="stopBtn">Stop</button>

    <script>
      const startButton = document.getElementById("startBtn");
      const stopButton = document.getElementById("stopBtn");

      startButton.addEventListener("click", () => {
        console.log("Start");
      });

      stopButton.addEventListener("click", () => {
        console.log("Stop");
      });
    </script>
```

```
</body>
</html>
```

Expected behavior:

```
Click Start → Start
Click Stop → Stop
```

27. Extra Challenge

Task:

Add a paragraph whose text changes when a button is clicked.

Solution:

```
<!DOCTYPE html>
<html>
  <body>
    <h1>Change Text</h1>

    <p id="message">Original text</p>
    <button id="changeBtn">Change Text</button>

    <script>
      const message = document.getElementById("message");
      const button = document.getElementById("changeBtn");

      button.addEventListener("click", () => {
        message.textContent = "The text has changed!";
        console.log("Text changed");
      });
    </script>
  </body>
</html>
```

Expected behavior:

Before click:

Original text

After click:

The text has changed!

Console:

Text changed

28. Full Sprint 1 Practice Project

This project combines the main Sprint 1 concepts.

```
<!DOCTYPE html>
<html>
  <body>
    <!-- HTML goal: create a small interactive page -->
    <h1>Sprint 1 Practice</h1>
    <p id="statusText">No button clicked yet.</p>

    <button id="helloBtn">Hello</button>
    <button id="learnBtn">Learning JavaScript</button>
    <button id="byeBtn">Goodbye</button>

    <script>
      // JavaScript goal: listen for button clicks and update the page
      console.log("Page loaded");

      const statusText = document.getElementById("statusText");
      const helloButton = document.getElementById("helloBtn");
      const learnButton = document.getElementById("learnBtn");
      const byeButton = document.getElementById("byeBtn");

      console.log("Buttons ready");

      helloButton.addEventListener("click", () => {
        console.log("Hello");
        statusText.textContent = "Hello button was clicked.";
      });
```

```

learnButton.addEventListener("click", () => {
  console.log("Learning JavaScript");
  statusText.textContent = "You are learning JavaScript.";
});

byeButton.addEventListener("click", () => {
  console.log("Goodbye");
  statusText.textContent = "Goodbye button was clicked.";
});
</script>
</body>
</html>

```

This project includes:

- HTML structure
- a heading
- a paragraph
- three buttons
- JavaScript inside a `<script>` tag
- comments
- semicolons
- `console.log()`
- `getElementById()`
- `addEventListener()`
- click events
- `textContent`
- visible page changes

29. Common Sprint 1 Mistakes

Mistake 1: Wrong ID

HTML:

```
<button id="runBtn">Run</button>
```

Wrong JavaScript:

```
const button = document.getElementById("runButton");
```

Problem:

`runButton` does not match `runBtn` .

Fix:

```
const button = document.getElementById("runBtn");
```

Mistake 2: Forgetting Quotes Around the ID

Wrong:

```
const button = document.getElementById(runBtn);
```

Correct:

```
const button = document.getElementById("runBtn");
```

The ID must be written as a string.

Mistake 3: Expecting `console.log()` to Show on the Page

Wrong expectation:

```
console.log("Hello");
```

Beginner thinks this will show on the webpage.

Correct understanding:

`console.log()` shows the message in the browser console, not on the page.

To change text on the page, use something like:

```
paragraph.textContent = "Hello";
```

Mistake 4: Forgetting to Open the Console

If you use `console.log()`, you need to open the browser console to see the output.

Usual path:

Right click → Inspect → Console

Mistake 5: Writing Code Before the Element Exists

If JavaScript tries to find a button before the browser has created that button, it may fail.

In Sprint 1 examples, the `<script>` tag is placed near the end of the `<body>`.

Example:

```
<button id="runBtn">Run</button>

<script>
  const button = document.getElementById("runBtn");
</script>
```

This is good because the button exists before JavaScript searches for it.

30. Key Vocabulary

HTML

The language used to structure web page content.

CSS

The language used to style web page content.

JavaScript

The language used to add behavior and interactivity to web pages.

Console

A developer area in the browser where messages and errors appear.

```
console.log()
```

A JavaScript tool that prints messages or values in the browser console.

Comment

A note in code that is ignored when the program runs.

Semicolon

A symbol used to mark the end of many JavaScript statements.

Statement

One instruction in JavaScript.

```
id
```

A unique name given to an HTML element.

```
document
```

The JavaScript object that represents the current web page.

```
getElementById()
```

A method used to find an HTML element by its ID.

Event

Something that happens on the page, such as a click.

Click Event

An event that happens when a user clicks an element.

```
addEventListener()
```

A method used to listen for an event and run code when that event happens.

Arrow Function

A short function syntax. In Sprint 1, it is used as the code that runs after a click.

`alert()`

A method that shows a popup message in the browser.

`textContent`

A property used to read or change the text inside an HTML element.

`null`

A value meaning “nothing.” If `getElementById()` cannot find an element, it returns `null`.

31. Sprint 1 Mental Model

Think of the process like this:

```
HTML creates the button.  
JavaScript finds the button.  
JavaScript listens for a click.  
The user clicks the button.  
JavaScript runs the code.  
The page responds.
```

Example:

```
<button id="runBtn">Run</button>
```

```
const button = document.getElementById("runBtn");  
  
button.addEventListener("click", () => {  
  console.log("Button clicked");  
});
```

Simple explanation:

1. Create a button.

2. Find the button.
 3. Wait for a click.
 4. Print a message after the click.
-

32. Sprint 1 Revision Questions

Try to answer these without looking at the notes.

1. What does HTML do?
 2. What does CSS do?
 3. What does JavaScript do?
 4. What does JavaScript add to a page that HTML and CSS do not add by themselves?
 5. What does `console.log()` do?
 6. Where do you see `console.log()` output?
 7. Why are comments useful?
 8. What symbol creates a single-line JavaScript comment?
 9. What does a semicolon do?
 10. What is an HTML `id`?
 11. Why should IDs be unique?
 12. What does `document.getElementById("runBtn")` do?
 13. What happens if JavaScript searches for the wrong ID?
 14. What does `null` mean in this beginner example?
 15. What is an event?
 16. What is a click event?
 17. What does `addEventListener("click", ...)` do?
 18. What does `alert()` do?
 19. What is the difference between `alert()` and `console.log()`?
 20. What does `textContent` do?
-

33. Sprint 1 Self-Check Answers

1. What does HTML do?

HTML creates the structure and content of the page.

2. What does CSS do?

CSS controls how the page looks.

3. What does JavaScript do?

JavaScript adds behavior and interactivity.

4. What does JavaScript add that HTML and CSS do not add by themselves?

JavaScript lets the page respond to actions, such as button clicks.

5. What does `console.log()` do?

It prints a message or value in the browser console.

6. Why are comments useful?

Comments explain code for people reading it. The browser ignores them.

7. What is the difference between structure, style, and behavior?

- Structure: HTML
 - Style: CSS
 - Behavior: JavaScript
-

8. What does `getElementById()` do?

It finds an HTML element using its ID.

9. What happens if the ID is wrong?

JavaScript cannot find the element and returns `null`.

10. What does `addEventListener("click", ...)` do?

It waits for a click and runs code when the click happens.

34. Final Sprint 1 Checklist

You are ready to move on from Sprint 1 when you can do these things:

- ☐ Explain the difference between HTML, CSS, and JavaScript.
 - ☐ Create a simple HTML page.
 - ☐ Add a heading, paragraph, and button.
 - ☐ Add JavaScript inside a `<script>` tag.
 - ☐ Use `console.log()` to print messages.
 - ☐ Open the browser console.
 - ☐ Add useful comments.
 - ☐ Use semicolons at the end of statements.
 - ☐ Give a button an ID.
 - ☐ Find the button using `document.getElementById()`.
 - ☐ Add a click listener using `addEventListener()`.
 - ☐ Show an alert when a button is clicked.
 - ☐ Change paragraph text using `textContent`.
 - ☐ Explain why a wrong ID causes an error.
 - ☐ Build Start and Stop buttons.
 - ☐ Build a button that changes visible text.
-

35. One-Sentence Summary

Sprint 1 teaches you that HTML creates the page, CSS styles the page, and JavaScript makes the page respond to actions like button clicks.